139. Word Break - Explanation

Description
Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of dictionary words.

You are allowed to reuse words in the dictionary an unlimited number of times. You may assume all dictionary words are unique.

Example 1:
Input: s = "neetcode", wordDict = ["neet","code"]
Output: true
Explanation: Return true because "neetcode" can be split into "neet" and "code".

Example 2:
Input: s = "applepenapple", wordDict = ["apple","pen","ape"]
Output: true
Explanation: Return true because "applepenapple" can be split into "apple", "pen" and "apple". Notice that we can reuse words and also not use all the words.

Example 3:
Input: s = "catsincars", wordDict = ["cats","cat","sin","in","car"]
Output: false

Constraints:
1 <= s.length <= 200
1 <= wordDict.length <= 100
1 <= wordDict[i].length <= 20
s and wordDict[i] consist of only lowercase English letters.

1. Recursion
Intuition
At every index i in the string, we want to decide:

Can the suffix starting at index i be segmented into valid dictionary words?
The recursive idea is:

Try every word in wordDict
If a word matches the string starting at position i
Recursively check whether the remaining substring (starting at i + len(word)) can also be broken successfully
If any path reaches the end of the string, the answer is True.

This is a classic decision-based recursion where:

Each index i represents a subproblem
Base case: reaching the end means a valid segmentation

Algorithm
Define a recursive function dfs(i):

If i == len(s), return True
For each word w in wordDict:
Check if w matches s[i : i + len(w)]
If it matches and dfs(i + len(w)) is True, return True
If no word leads to a valid segmentation, return False
Start recursion from index 0

```
public class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        return dfs(s, wordDict, 0);
    }

    private boolean dfs(String s, List<String> wordDict, int i) {
        if (i == s.length()) {
            return true;
        }

        for (String w : wordDict) {
            if (i + w.length() <= s.length() &&
                s.substring(i, i + w.length()).equals(w)) {
                if (dfs(s, wordDict, i + w.length())) {
                    return true;
                }
            }
        }
        return false;
    }
}
```

Time & Space Complexity
Time complexity: O (t * m^n)
Space complexity: O(n)
Where n is the length of the string s, m is the number of words in wordDict and t is the maximum length of any word in wordDict.

2. Recursion (Hash Set)
Intuition
This version improves the brute-force recursion by optimizing word lookup.

Instead of trying every word from wordDict at each index, we:

Fix a starting index i
Try all possible substrings s[i : j+1]
Check if the substring exists in a Hash Set (O(1) lookup)
If a valid word is found:

Recursively check whether the remaining suffix starting at j + 1 can be segmented

The Key idea:

If we can split the string at any valid word boundary and the rest is solvable, then the whole string is solvable.

Algorithm
Convert wordDict into a hash set wordSet for fast lookup
Define a recursive function dfs(i):
If i == len(s), return True
For every j from i to len(s) - 1:
If s[i : j + 1] is in wordSet
If dfs(j + 1) is True, return True
If no split works, return False
Start recursion from index 0

```
public class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        HashSet<String> wordSet = new HashSet<>(wordDict);

        return dfs(s, wordSet, 0);
    }

    private boolean dfs(String s, HashSet<String> wordSet, int i) {
        if (i == s.length()) {
            return true;
        }

        for (int j = i; j < s.length(); j++) {
            if (wordSet.contains(s.substring(i, j + 1))) {
                if (dfs(s, wordSet, j + 1)) {
                    return true;
                }
            }
        }
        return false;
    }
}
```

Time & Space Complexity
Time complexity: O((n * 2^n)+m)
Space complexity: O(n+(m * t))

Where n is the length of the string s and m is the number of words in wordDict.

3. Dynamic Programming (Top-Down)
Intuition
This is an optimized version of recursion using memoization.

The key observation:

While recursively checking splits, the same index i is reached many times
The result of dfs(i) (can suffix s[i:] be segmented?) never changes
So we cache the result for each index:

If dfs(i) was already computed, reuse it
This avoids recomputing exponential subtrees
In short:

Convert exponential recursion into linear states using memoization.

Algorithm
Use a hash map memo where:
memo[i] = True/False means whether s[i:] can be segmented
Base case: memo[len(s)] = True
Define dfs(i):
If i is in memo, return memo[i]
For each word w in wordDict:
If s[i : i + len(w)] == w
Recursively call dfs(i + len(w))
If it returns True, store memo[i] = True and return True
If no word leads to a valid split:
Store memo[i] = False
Return dfs(0)

```
public class Solution {
    private Map<Integer, Boolean> memo;

    public boolean wordBreak(String s, List<String> wordDict) {
        memo = new HashMap<>();
        memo.put(s.length(), true);
        return dfs(s, wordDict, 0);
    }

    private boolean dfs(String s, List<String> wordDict, int i) {
        if (memo.containsKey(i)) {
            return memo.get(i);
        }

        for (String w : wordDict) {
            if (i + w.length() <= s.length() &&
                s.substring(i, i + w.length()).equals(w)) {
                if (dfs(s, wordDict, i + w.length())) {
                    memo.put(i, true);
                    return true;
                }
```

```
            }
        }
        memo.put(i, false);
        return false;
    }
}
```

Time & Space Complexity
Time complexity: O(n * m * t)
Space complexity: O(n)
Where n is the length of the string s, m is the number of words in wordDict and t is the maximum length of any word in wordDict.

4. Dynamic Programming (Hash Set)
Intuition
This approach is a top-down dynamic programming solution with pruning.

Key ideas:

Checking every possible substring is expensive.
A word can only be as long as the maximum word length in wordDict.
Use a Hash Set for O(1) word lookup.
Use memoization so each index in the string is solved only once.
So we:

Limit how far we try to split from each index
Cache results for indices to avoid repeated work
This turns exponential recursion into efficient DP.

Algorithm
Convert wordDict into a hash set wordSet
Compute t = maximum length of any word in wordDict
Use a memo map where memo[i] means:
Can substring s[i:] be segmented?
Define dfs(i):
If i is in memo, return memo[i]
If i == len(s), return True
For j from i to min(len(s), i + t) - 1:
If s[i : j + 1] is in wordSet
If dfs(j + 1) is True, store and return True
If no valid split works:
Store memo[i] = False
Return dfs(0)

```
public class Solution {
    private HashSet<String> wordSet;
    private Boolean[] memo;
    private int t;
```

```
    public boolean wordBreak(String s, List<String> wordDict) {
        wordSet = new HashSet<>(wordDict);
        memo = new Boolean[s.length()];
        t = 0;
        for (int i = 0; i < wordDict.size(); i++) {
            t = Math.max(t, wordDict.get(i).length());
        }
        return dfs(s, 0);
    }

    private boolean dfs(String s, int i) {
        if (i == s.length()) {
            return true;
        }
        if (memo[i] != null) {
            return memo[i];
        }

        for (int j = i; j < Math.min(i + t, s.length()); j++) {
            if (wordSet.contains(s.substring(i, j + 1))) {
                if (dfs(s, j + 1)) {
                    memo[i] = true;
                    return true;
                }
            }
        }
        memo[i] = false;
        return false;
    }
}
```

Time & Space Complexity
Time complexity: O((t^2 * n)+m)
Space complexity: O(n+(m * t))

Where n is the length of the string s, m is the number of words in wordDict and t is the maximum length of any word in wordDict.

5. Dynamic Programming (Bottom-Up)

Intuition

This is a bottom-up dynamic programming approach.

Instead of trying to split the string recursively, we solve the problem from the end of the string toward the start.

Key idea:

dp[i] means whether the substring s[i:] can be segmented
If we know the answer for future positions, we can decide the current one
We reuse already computed results → no recursion, no stack overhead

Algorithm

Create a boolean array dp of size len(s) + 1
dp[i] = True if s[i:] can be segmented
Base case:
dp[len(s)] = True (empty string is valid)
Iterate i from len(s) - 1 down to 0:
For each word w in wordDict:
If s[i : i + len(w)] == w
Set dp[i] = dp[i + len(w)]
If dp[i] becomes True, break early
Return dp[0]

```
public class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        boolean[] dp = new boolean[s.length() + 1];
        dp[s.length()] = true;

        for (int i = s.length() - 1; i >= 0; i--) {
            for (String w : wordDict) {
                if ((i + w.length()) <= s.length() &&
                     s.substring(i, i + w.length()).equals(w)) {
                    dp[i] = dp[i + w.length()];
                }
                if (dp[i]) {
                    break;
                }
            }
        }

        return dp[0];
    }
}
```

Time & Space Complexity

Time complexity: O(n * m * t)
Space complexity: O(n)

Where n is the length of the string s, m is the number of words in wordDict and t is the maximum length of any word in wordDict.

6. Dynamic Programming (Trie)

Intuition

The normal DP checks every word at every index, which can waste time comparing strings again and again.

A Trie stores all dictionary words like a prefix tree, so from any starting index i in s, we can walk forward character-by-character and quickly know:

whether the current prefix matches some dictionary word path
and when we hit a complete word (is_word = True)
We still use DP:

dp[i] = can we break the suffix s[i:] into dictionary words?
If from i we can reach some j where s[i..j] is a word, then dp[i] = dp[j+1]
Trie helps us find valid words starting at i efficiently.

Algorithm

Build a Trie from all words in wordDict.
Create a boolean DP array dp of size n + 1 where n = len(s).
dp[n] = True (empty suffix is always valid)
Let t be the maximum word length in the dictionary (use it as a bound).
Fill DP from right to left:
For each index i from n down to 0:
Try all end positions j from i to min(n-1, i+t-1):
If s[i..j] is a word in the Trie, set dp[i] = dp[j+1]
If dp[i] becomes True, stop early for this i
Return dp[0].

```
class TrieNode {
    HashMap<Character, TrieNode> children;
    boolean isWord;

    public TrieNode() {
        children = new HashMap<>();
        isWord = false;
    }
}

class Trie {
    TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            node.children.putIfAbsent(c, new TrieNode());
            node = node.children.get(c);
        }
```

```
        node.isWord = true;
    }

    public boolean search(String s, int i, int j) {
        TrieNode node = root;
        for (int idx = i; idx <= j; idx++) {
            if (!node.children.containsKey(s.charAt(idx))) {
                return false;
            }
            node = node.children.get(s.charAt(idx));
        }
        return node.isWord;
    }
}

public class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        Trie trie = new Trie();
        for (String word : wordDict) {
            trie.insert(word);
        }

        int n = s.length();
        boolean[] dp = new boolean[n + 1];
        dp[n] = true;

        int maxLen = 0;
        for (String word : wordDict) {
            maxLen = Math.max(maxLen, word.length());
        }

        for (int i = n - 1; i >= 0; i--) {
            for (int j = i; j < Math.min(n, i + maxLen); j++) {
                if (trie.search(s, i, j)) {
                    dp[i] = dp[j + 1];
                    if (dp[i]) break;
                }
            }
        }

        return dp[0];
    }
}
```

Time & Space Complexity
Time complexity:
O((n * t^2)+m)
Space complexity: O(n+(m * t))

Where n is the length of the string s, m is the number of words in wordDict and t is the maximum length of any word in wordDict.